ใบงาน Week 06 การปรับตัวและปฏิสัมพันธ์

สำหรับนักศึกษา | ออกแบบ state machine และตรวจการตอบสนองด้วย movement และ RGB

ชื่อและรหัส นาย บารมี สำเภาทอง 6752500380

กลุ่ม มินเนี่ยนและผองเพื่อน

วันที่ 8/11/2569

1 เป้าหมายและเตรียมแล็บ

ออกแบบตัวควบคุมให้รับ button press, touch และ person approaching จากระยะห่าง แล้วแสดง normal, interested, happy, afraid และ sleepy ด้วยท่าทางและ RGB พร้อมหลักฐานว่าเริ่มตอบสนองภายใน 1 วินาทีจากต้นทาง interaction

อธิบายให้ได้ว่า state ที่เปลี่ยนตามกฎคงที่เป็นการปรับพฤติกรรม ส่วน learning ต้องมีพารามิเตอร์หรือ policy เปลี่ยนจากประสบการณ์ แล็บหลักนี้ใช้ finite state machine และยังไม่ได้ฝึกโมเดลอารมณ์

ค่าคงที่ เกณฑ์ และ mapping ในแล็บนี้เป็น proposed course model คำว่า emotion หมายถึงชื่อ state เพื่อสื่อพฤติกรรมของหุ่นยนต์ ไม่ใช่การรับรองว่าหุ่นยนต์มีความรู้สึก

เปิดแพ็กเกจนักศึกษา

แตกไฟล์ Week06_Student_Package.zip แล้วเปิด terminal ในโฟลเดอร์ Week06_Student_Package จากนั้นเข้า lab06 คำสั่งต่อไปใช้ path ภายในแพ็กเกจ

cd lab06

python3 --version

python3 lab06.py --help

python3 check_submission.py --contract group

ใช้ Python 3.10 ขึ้นไปและ standard library เท่านั้น Windows ใช้ py -3 แทน python3 ได้ Starter มี TODO เพื่อให้เติม policy จึงควรตรวจพบส่วนที่ยังไม่ครบในครั้งแรก

ก่อนต่อหุ่นยนต์จริง

จำลองก่อนเสมอ จากนั้นผู้สอนต้องสอบเทียบศูนย์ หน่วย ทิศทาง ขีดจำกัดของทุกแกน และความเร็วกับอุปกรณ์จริง เริ่มจากท่ากลางและการเคลื่อนไหวขนาดเล็ก เตรียมวิธีหยุดฉุกเฉินที่เข้าถึงได้

การตรวจเริ่มงาน	บันทึก
Python version และผู้รับผิดชอบแต่ละหน้าที่	Python 3.14.6, Windows PowerShell; ผู้จัดทำรับผิดชอบแก้ FSM Policy ทดสอบและวิเคราะห์ข้อมูล
วิธีหยุดและผู้ดูแลพื้นที่เคลื่อนไหว	ทดลองด้วย Offline Simulation ไม่มีการขยับอุปกรณ์จริง หากทดสอบฮาร์ดแวร์ต้องกำหนดผู้ดูแลพื้นที่และวิธีหยุดฉุกเฉินก่อน
หลักฐานว่าค่าสอบเทียบใช้ได้กับอุปกรณ์นี้	ยังไม่ได้ยืนยันกับฮาร์ดแวร์จริง มีเพียงผล Simulation และ Checker ผ่าน 40/40 กรณี ต้องตรวจขอบเขตมุม ความเร็ว การชน และกำลังของอุปกรณ์ก่อนใช้งาน

Prediction ก่อนรัน เมื่อ near กับ touch มาพร้อมกัน หุ่นยนต์ควรทำอะไร และเพราะเหตุใด

คาดว่าเมื่อเกิด near และ touch พร้อมกัน โคมไฟควรเข้าสู่สถานะ afraid แทน happy เพราะ near มี Priority สูงกว่า touch เพื่อให้ระบบตอบสนองต่อวัตถุที่เข้าใกล้ก่อน หากโคมไฟอยู่ใน afraid แล้วได้รับ touch ระบบควรคงสถานะเดิมตาม Safety Guard จนกว่าจะเกิด clear หรือ fault เพื่อกลับสู่ normal

---

2 ตาราง transition และเงื่อนไข

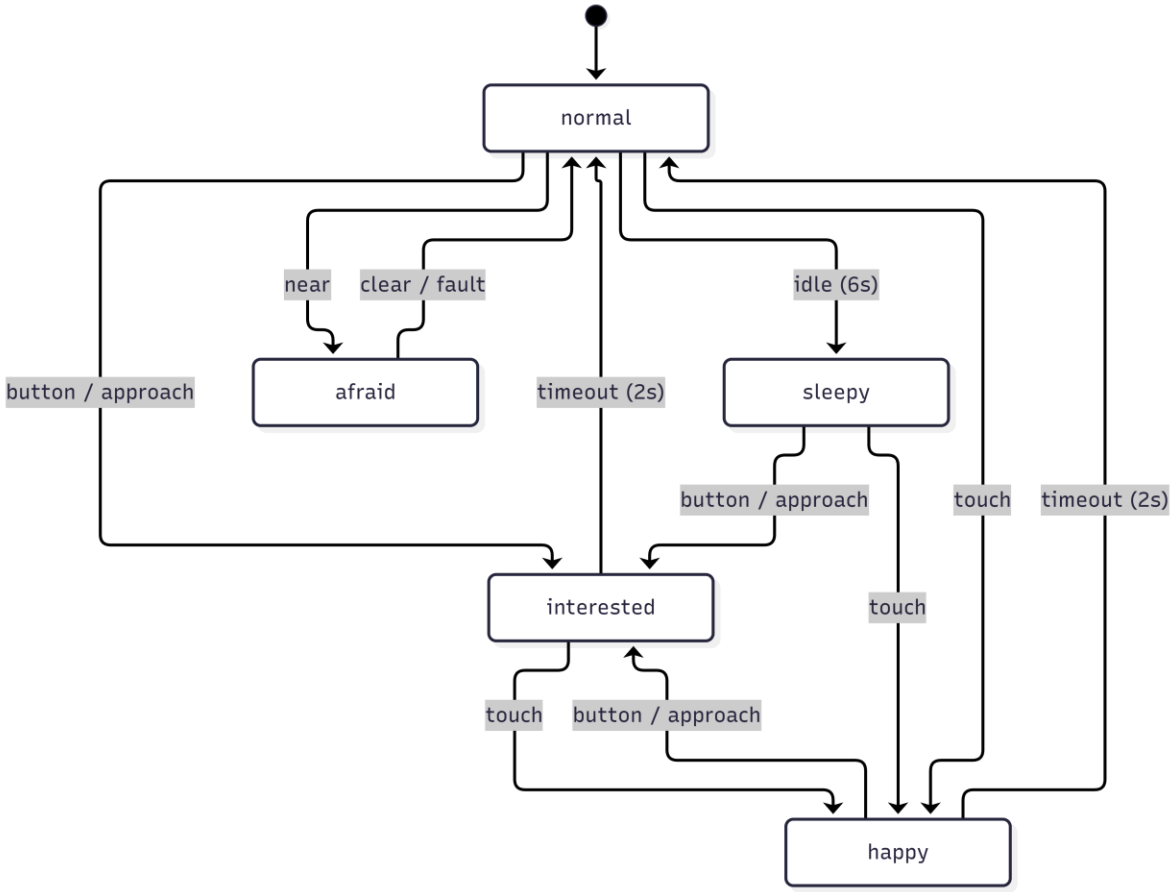
อ่าน README แล้วเติม next state ทุกช่องด้วยชื่อ state หรือ hold กลุ่มออกแบบ positive interaction mapping ได้โดยรักษา safety recovery และต้องเข้าถึงครบ 5 states ตาม group contract พร้อมอธิบาย priority

current event →	fault	near	clear	touch	button	approach	timeout	idle
normal	normal	afraid	normal	happy	interested	interested	_____	sleepy
interested	normal	afraid	normal	happy	interested	interested	normal	_____
happy	normal	afraid	normal	happy	interested	interested	normal	_____
afraid	normal	afraid	normal	_____	_____	_____	_____	_____
sleepy	normal	afraid	normal	happy	interested	interested	_____	_____

fault คือ input ใช้ไม่ได้; near คือระยะอันตรายที่กำหนด; clear คือพ้นเขต near; approach คือคนเข้ามาในเขต interaction; timeout และ idle เกิดจากเวลา เขียนให้ชัดว่า event ที่ไม่ใช้ใน state นั้นให้ hold หรือเปลี่ยน state

ข้อกำหนดออกแบบ	คำตอบของกลุ่ม
ลำดับ priority เมื่อหลาย event มาพร้อมกัน	fault > near > clear > touch > button > approach > timeout > idle เมื่อมีหลายเหตุการณ์พร้อมกัน ระบบเลือกเหตุการณ์ที่มีลำดับความสำคัญสูงสุดเพียงรายการเดียว ส่วนเหตุการณ์ที่เหลือบันทึกสถานะเป็น discarded_priority
Entry และ exit ของ afraid	Entry: เมื่อเกิด near (C1 ตรวจพบระยะ ≤ 0.20 m) ระบบเปลี่ยนเป็น afraid Exit: เมื่อเกิด clear ระบบกลับเป็น normal หรือเมื่อเกิด fault จะกลับเป็น normal ตาม Safety Policy ระหว่างอยู่ใน afraid เหตุการณ์ touch, button, approach, timeout และ idle จะไม่ทำให้เปลี่ยนสถานะ
Timeout ของ interested และ happy	เมื่อโคมไฟอยู่ในสถานะ interested หรือ happy ครบ 2 วินาทีตามเวลาของแบบจำลอง จะเกิดเหตุการณ์ timeout และเปลี่ยนกลับเป็น normal โดยอัตโนมัติ หากมีเหตุการณ์ที่มี Priority สูงกว่าเข้ามาก่อน ระบบจะจัดการเหตุการณ์นั้นตามลำดับความสำคัญ
Inactivity ที่ทำให้เข้าสู่ sleepy	เมื่อโคมไฟอยู่ในสถานะ normal และไม่มีปฏิสัมพันธ์ต่อเนื่องครบ 6 วินาที ระบบจะสร้างเหตุการณ์ idle ทำให้เปลี่ยนเป็น sleepy เมื่อได้รับ button หรือ approach จะเข้าสู่ interested และเมื่อได้รับ touch จะเข้าสู่ happy
Debounce และ hysteresis ป้องกันอะไร	Debounce ช่วยกรองการเด้งของสัญญาณปุ่มกดและ Touch โดย C1 กำหนดให้สัญญาณคงที่อย่างน้อย 0.06 วินาทีก่อนยอมรับเหตุการณ์ Hysteresis ช่วยลดการสลับสถานะจากระยะทางที่มี Noise โดยใช้เกณฑ์เข้าและออกต่างกัน เช่น Near เข้าเมื่อระยะ ≤ 0.20 m และออกเมื่อระยะ ≥ 0.28 m ผลการทดลอง 3 Seeds พบว่า C1 ลด State Transitions จาก 202 เหลือ 28 ครั้ง หรือลดลงประมาณ 86.1% เมื่อเทียบกับ C0

วาด state diagram ให้ลูกศรมีชื่อ event และ guard ระบุทุกทางกลับสู่ normal



แผนภาพแสดงการเปลี่ยนสถานะหลักของ FSM โดยเหตุการณ์ near สามารถเข้าสู่ afraid จากทุกสถานะ และ clear/fault สามารถกลับสู่ normal ได้จากทุกสถานะ ขณะอยู่ใน afraid ระบบจะไม่ตอบสนองต่อ touch, button, approach, timeout และ idle รายละเอียดครบทุกกรณีแสดงใน Transition Table

ตรวจว่าตารางกับ diagram ให้คำตอบเดียวกันทุกช่องก่อนเขียนโค้ด

ตรวจสอบแล้ว แผนภาพแสดงการเปลี่ยนสถานะหลักสอดคล้องกับ Transition Table ส่วนกรณี Hold และการเปลี่ยนสถานะจากทุก State แสดงรายละเอียดครบในตาราง โดยทดสอบ Baseline Contract ผ่าน 40/40 กรณี

3 การแสดง state และงบเวลา

ออกแบบการแสดงออกให้ครอบคลุมเอียงตัว ยืดหรือหดท่าทาง กลับสู่ท่ากลาง และกัมหรือเงย ใช้ชื่อการเคลื่อนไหวและ RGB ที่สังเกตได้ หลีกเลี่ยงการใช้สีเป็นหลักฐานเพียงอย่างเดียว

State	Movement และเหตุผล	RGB 0 ถึง 255	Entry หรือ recovery
normal	tilt = 0, extend = 0, nod = 0 rad เป็นท่ากลาง ไม่เอียง ไม่ยืดหรือกัม	(60, 90, 140)	สถานะเริ่มต้น หรือกลับจาก clear/fault และ timeout ของ interested/happy
interested	tilt = 0.22, extend = 0.18, nod = 0.06 rad เอียงและยืดตัวไปด้านหน้าเล็กน้อย เพื่อแสดงความสนใจ	(255, 180, 0)	เข้าสู่สถานะเมื่อเกิด button หรือ approach และกลับเป็น normal เมื่อเกิด timeout ครบ 2 วินาที
happy	tilt = 0.10, extend = 0.10, nod = 0.18 × sin(2πt) rad ขยับศีรษะขึ้นลงเป็นจังหวะ 1 Hz เพื่อแสดงความดีใจ	(40, 220, 80)	เข้าสู่ happy เมื่อเกิด touch ยกเว้นขณะ afraid และกลับสู่ normal เมื่อเกิด timeout หลัง 2 วินาที
afraid	tilt = −0.22, extend = −0.22, nod = −0.16 rad ใช้มุมติดลบเพื่อแสดงท่าทางหดและกัมหลบ	(230, 30, 30)	เข้าสู่ afraid เมื่อเกิด near และกลับสู่ normal เมื่อเกิด clear หรือ fault
sleepy	tilt = 0, extend = −0.15, nod = −0.20 rad ลดการยืดตัวและกัมศีรษะเพื่อแสดงอาการง่วง	(50, 40, 100)	เข้าสู่ sleepy เมื่อเกิด idle ขณะอยู่ใน normal และกลับเป็น interested เมื่อได้รับ button หรือ approach

Simulator ใช้ q = [tilt, extend, nod] หน่วย rad เป็นแกนแสดงท่าทางเชิงนามธรรม ต้องสอบเทียบก่อนนำไปแปลงเป็นคำสั่ง joint ของหุ่นยนต์จริง Bounds และ slew ใน config เป็น proposed course model

กำหนดจุดเริ่มและจุดจบของการวัด

ต้นทางคือเวลาผู้ใช้กด สัมผัส หรือข้ามเกณฑ์ระยะ ไม่ใช่เวลาที่ controller ได้รับ event แยก first motion, RGB change และ both ก่อนตัดสินว่า ≤ 1 s โดย both ใช้ค่าที่ช้ากว่าเมื่อวัดได้ครบสองช่องทาง

ส่วนที่ใช้เวลา	งบ s	วิธีวัด
Sensor และ transport	0.10	บันทึกเวลาที่ Input เริ่มเกิด (source_time_s) และเวลาที่ข้อมูลเซนเซอร์มาถึงระบบควบคุม แล้วนำเวลาทั้งสองมาหาผลต่าง
Filter และ decision	0.20	ดเวลาตั้งแต่เหตุการณ์เริ่มเกิด (source_time_s) ผ่านการกรองสัญญาณ (recognized_time_s) จน FSM ตัดสินใจ (decision_time_s) โดยแยกเวลารอ Debounce ออกจากเวลาตัดสินใจ
Queue และ actuator หรือ RGB	0.30	บันทึกเวลาเข้าคิว (queued_time_s) เวลาเริ่มนำคำสั่งไปใช้ (applied_time_s) และเวลาเริ่มเปลี่ยนท่าทาง (first_motion_s) กับสีไฟ (first_rgb_s) แล้วหาผลต่าง เพื่อวัดความล่าช้าในขั้น Queue และ Output

ส่วนที่ใช้เวลา	งบ s	วิธีวัด
รวมและส่วนเผื่อถึง 1 s	1.00 (ใช้ 0.60 + เผื่อ 0.40)	วัดเวลาตั้งแต่ source_time_s จนทั้ง Motion และ RGB เริ่มเปลี่ยนจริง โดยใช้เวลาที่ช้ากว่าของทั้งสองช่องทางเป็นจุดสิ้นสุด ต้องไม่เกิน 1.00 s สำหรับเหตุการณ์ที่คาดหวังการตอบสนอง

ถ้า output ไม่เปลี่ยนหรือยังไม่เกิดก่อนจบ trial จะบันทึกอย่างไร และทำไมไม่ใส่ 0 s

หาก Motion หรือ RGB ไม่เกิดการเปลี่ยนแปลงก่อนจบ Trial ให้รายงาน Latency เป็น N/A ไม่ใช่ 0 s พร้อมระบุสาเหตุ เช่น policy_held, no_state_change, preempted_before_apply หรือ censored_end เพราะ 0 s หมายถึงตอบสนองทันที แต่ N/A หมายถึงไม่มีค่าที่สามารถวัดได้

---

4 Emotion controller lab 50 นาที

นาทีในกิจกรรม	ทำและตรวจ
0 ถึง 5	Setup และอ่าน contract
5 ถึง 26	เติม student_policy.py ตาม prediction
26 ถึง 34	Checker และแก้หนึ่งรอบ
34 ถึง 42	C0 C1 อย่างละ 3 paired runs
42 ถึง 50	อ่าน log และบันทึก finding

แก้เฉพาะ transition(current_state, event) ใน student_policy.py คืนชื่อ state ที่อนุญาตหรือ None เพื่อคง state อย่าใส่ sleep หรือติดต่อ sensor และ actuator ใน policy Engine จัด filter, scheduler, timeout และ bounds

python3 check_submission.py --contract group

python3 lab06.py --policy student --condition all --repeats 3 --output-dir results/my_run

ใช้ --contract baseline เมื่อตาม reference mapping เปิด results/my_run/report.html ใน browser ใช้ output directory ใหม่เมื่อแก้ policy C0 raw กับ C1 guarded ต้องใช้ policy mapping safety และ seed คู่เดียวกัน

จุดตรวจ	ค่าที่อ่านจาก raw CSV หรือ event CSV
near และ touch พร้อมกันที่ 7 s	จาก C1 Seed 43 เมื่อเกิด near ระบบเข้าสู่ afraid และเมื่อได้รับ touch_conflict ที่เวลา 7.06 s ขณะอยู่ในสถานะ afraid ระบบคงสถานะเดิม (policy_held) ตาม Safety Guard ไม่เปลี่ยนเป็น happy โดย near มี Priority สูงกว่า touch
clear ที่ 9 s	จาก Event CSV ของ Seed 43 ที่เวลา 9.00 s พบว่า C0 ได้รับ clear_02 ขณะอยู่ใน normal จึงมีสถานะ no_state_change และ Both Latency เป็น N/A ส่วน C1 เปลี่ยนจาก afraid → normal มีสถานะ responded และ Both Latency = 0.04 s แสดงว่าการตอบสนองขึ้นอยู่กับสถานะก่อนเกิดเหตุการณ์
invalid ที่ 11 s และ stale ที่ 12 s	จาก C1 Seed 43 พบว่า invalid_01 ที่ 11.00 s และ stale_01 ที่ 12.00 s ถูกตรวจจับเป็นเหตุการณ์ fault ทั้งคู่ ระบบใช้ Fallback ไปยัง normal แต่เนื่องจากสถานะเดิมเป็น normal จึงมี Status เป็น no_state_change และ Both Latency เป็น N/A ไม่ใช่ 0 s ผลนี้ยืนยันการจัดการ Fault ตามกฎของแบบจำลอง แต่ยังไม่รับรอง Hardware Safety
button ที่ 18 s หลัง inactivity	จาก C1 Seed 43 ระบบเข้าสู่ sleepy เมื่อเกิด idle ที่ 15.00 s ต่อมาได้รับ button_02 ที่ 18.00 s หลังผ่าน Debounce จึงเปลี่ยนจาก sleepy → interested ที่ 18.06 s และเกิด Motion/RGB Onset ที่ 18.10 s ได้ Both Latency = 0.10 s ซึ่งผ่านเกณฑ์ ≤ 1 s ของแบบจำลอง
เหตุผลที่ state ไม่เปลี่ยนหนึ่งกรณี	จาก Event CSV ของ C1 Seed 43 พบว่า touch_conflict ที่ 7.06 s มีสถานะ policy_held เพราะโคมไฟอยู่ใน afraid จึงไม่เปลี่ยนเป็น happy ตาม Safety Guard ส่วน invalid_01 ที่ 11.00 s และ stale_01 ที่ 12.00 s มีสถานะ no_state_change เพราะระบบอยู่ใน normal อยู่แล้ว จึงไม่มีการเปลี่ยนสถานะหรือ Both Onset ใหม่ โดยต้องรายงาน Latency เป็น N/A ไม่ใช่ 0 s

เติม timeline จาก run ของตนเอง บันทึก input ต้นทางและเวลา decision แยกกัน

source s	event	state ก่อน	decision s	state หลัง
9.00	clear	afraid	9.00	normal
11.00	fault (invalid)	normal	11.00	normal
15.00	idle	normal	15.00	sleepy
18.00	button	sleepy	18.06	interested

Finding ที่ข้อมูลรองรับและชื่อไฟล์หรือคอลัมน์ที่ใช้อ้าง

จากการทดลอง C0/C1 จำนวน 3 Seeds พบว่า C1 ลด State Transitions จาก 202 เหลือ 28 ครั้ง คิดเป็น 86.1% และตอบสนอง Primary Inputs ครบ 21/21 ครั้ง ขณะที่ C0 ตอบสนอง 19/21 ครั้ง แสดงว่า Debounce และ Hysteresis ช่วยลดการสลับสถานะจากสัญญาณรบกวนในแบบจำลองนี้

หลักฐาน: results/completed_01/metrics.csv คอลัมน์ state_transition_count, primary_pass_1s_both_count และ primary_expected_count

---

5 RGB และ response time 30 นาที

นาทีในกิจกรรม	ทำและตรวจ
0 ถึง 10	ตรวจ movement ทั้ง 4 แบบกับสีของทุก state
10 ถึง 20	อ่าน source, decision, applied และ output onset
20 ถึง 30	สรุป latency แยก motion RGB both และกรณีไม่วัดได้

ประกาศ expected input IDs ก่อนรัน ใช้ source_outcomes ใน metrics.json รวม latency และ misses ตามชุดของกลุ่มเอง ค่า primary_* ของ runner ใช้ baseline 7 IDs เสมอ เก็บทุก source พร้อม status โดยไม่เติม missing เป็น 0 s

Input และ condition	motion s	RGB s	both s	status หรือเหตุผล
button / C0	0.04	0.04	0.04	responded
button / C1	0.14	0.14	0.14	responded
touch / C0	0.04	0.04	0.04	responded
touch / C1	0.10	0.10	0.10	responded
approach / C0	0.04	0.04	0.04	responded
approach / C1	0.04	0.04	0.04	responded

สรุปทุก repeat	C0 raw	C1 guarded
ทุก source และจำนวน both ที่วัดได้	30 / 19	30 / 21
Expected set ของกลุ่ม ผ่าน ≤ 1 s และ misses	19 / 2	21 / 0
Motion missing และ RGB missing	11 / 11	9 / 9
No change held filtered หรือ invalid	no_state_change = 7, filtered = 1, discarded_priority = 2, preempted = 1	no_state_change = 6, policy_held = 3

1 Command queued เวลาใด แล้ว actual simulated output เปลี่ยนเวลาใด ต่างกันเพราะอะไร

จาก C1 Seed 43 เหตุการณ์ clear_02 คำสั่งถูกส่งเข้าคิวที่เวลา 9.00 s และถูกนำไปใช้ที่ 9.04 s ซึ่งเป็นเวลาเดียวกับที่ Motion และ RGB เริ่มเปลี่ยนแปลง มีความล่าช้า 0.04 s เนื่องจาก Scheduler Delay ที่กำหนดไว้ในแบบจำลอง ไม่ใช่ความล่าช้าของฮาร์ดแวร์จริง

---

2 C1 ลดปัญหาใดจาก C0 และเสียเวลาเพิ่มเท่าไร ใช้หลักฐานแทนคำว่าเร็วหรือเสถียร

C1 ใช้ Debounce และ Hysteresis เพื่อลดการเปลี่ยนสถานะบ่อยครั้งจากสัญญาณเด้งและ Noise โดยผลทดลอง 3 Seeds พบว่า State Transitions ลดจาก 202 เหลือ 28 ครั้ง (ลดลง 86.1%) และ Primary Inputs ที่ตอบสนองภายใน 1 s เพิ่มจาก 19/21 เป็น 21/21 ครั้ง

อย่างไรก็ตาม ใน Seed 43 ค่า Button Latency เพิ่มจาก 0.04 เป็น 0.14 s (+0.10 s) และ Touch เพิ่มจาก 0.04 เป็น 0.10 s (+0.06 s) ขณะที่ Approach เท่ากันที่ 0.04 s จึงเป็นข้อแลกเปลี่ยนระหว่างการกรองสัญญาณกับเวลาตอบสนองของแบบจำลอง

---

3 Simulator ให้หลักฐานเวลาในแบบจำลอง ส่วนการรับรองอุปกรณ์จริงต้องเพิ่ม measurement ใด

Simulator บันทึกเวลาตั้งแต่เกิด Input จนถึง Motion และ RGB Onset ภายในซอฟต์แวร์ได้ แต่ยังไม่ใช่เวลาตอบสนองของหุ่นยนต์จริง หากต้องการยืนยันผล ต้องวัดเวลา Input จากเซนเซอร์จริง วัดการเริ่มเคลื่อนไหวด้วยกล้องความเร็วสูงหรือ Encoder และวัดการเริ่มเปลี่ยนแสงด้วย Photodiode โดยใช้นาฬิกาที่ซิงโครไนซ์กัน พร้อมระบุความละเอียดและความคลาดเคลื่อนของการวัด รวมถึงตรวจแรงบิด การชน และระบบหยุดฉุกเฉินก่อนรับรองความปลอดภัย

---

6 งานส่งและ exit quiz

ส่ง policy ของกลุ่มพร้อมรายงาน 1 ถึง 2 หน้า แสดง prediction, transition diagram, expression mapping และผลที่วัดได้ หากข้อมูลไม่พอให้ระบุ N/A พร้อมเหตุผล ไม่แทนด้วยค่า 0

รายการตรวจงานส่ง	ครบ
student_policy.py และตาราง 5 states กับ 8 events	____
Diagram และ mapping ครบ 3 mandatory inputs กับ 4 movements	____
Raw CSV event CSV config metrics และ manifest ของทุก condition	____
คำสั่งรัน Python version seed repeats และ output directory	____
Latency motion RGB both แยก source กับ command พร้อม misses	____
Finding ข้อจำกัด และแผนสอบเทียบ; hardware ใช้ผลสอบเทียบจริง	____

เกณฑ์ประเมิน 20 คะแนน

หมวด	คะแนน
FSM diagram transition guard และ recovery	4
Movement RGB contract และแผนสอบเทียบ	4
Policy checker fault และ priority	4
C0 C1 อย่างน้อย 3 คู่ CSV config และ metrics	4
Latency misses และข้อจำกัดของหลักฐาน	4

Exit quiz 10 นาที

1 State machine ช่วยตรวจความขัดแย้งของ if–else ได้อย่างไร

State Machine กำหนดสถานะ เหตุการณ์ และเงื่อนไขการเปลี่ยนสถานะไว้อย่างชัดเจน ทำให้ตรวจสอบได้ว่าแต่ละเหตุการณ์ควรเปลี่ยนไปสถานะใด ต่างจาก if–else ที่กระจายหลายจุดและอาจเกิดเงื่อนไขขัดแย้งกันได้ โดย FSM ใช้ Transition Table และ Event Priority เพื่อกำหนดผลลัพธ์ที่แน่นอน และสามารถทดสอบครบทุกกรณีได้

---

2 ทำไม afraid ต้องมีทั้ง entry, exit และ priority เมื่อ near กับ touch พร้อมกัน

afraid ต้องมี Entry เพื่อกำหนดว่าจะเข้าสู่สถานะกลัวเมื่อเกิด near และมี Exit เพื่อให้กลับสู่ normal เมื่อเกิด clear หรือ fault ไม่ให้ระบบค้างอยู่ในสถานะกลัวตลอดเวลา เมื่อ near และ touch ถูกตรวจพบพร้อมกัน ต้องใช้ Priority โดยกำหนดให้ near สำคัญกว่า touch เพื่อให้ระบบเข้าสู่ afraid แทน happy และระหว่างอยู่ใน afraid จะใช้ Safety Guard ป้องกันไม่ให้ touch เปลี่ยนอารมณ์โดยไม่เหมาะสม

---

3 ยกตัวอย่างส่วนที่ต้องเปลี่ยนจากประสบการณ์จึงจะเรียกว่า learning และหนึ่งงานเสริมที่อยู่นอกแล็บหลัก

ระบบ FSM ใน Lab 6 ยังไม่ถือว่าเป็น Learning เพราะเปลี่ยนสถานะตามกฎที่กำหนดไว้ล่วงหน้า โดยไม่มีการเรียนรู้หรือปรับพารามิเตอร์จากประสบการณ์ หากต้องการให้เกิด Learning อาจเพิ่มระบบบันทึกความชอบของผู้ใช้และปรับพารามิเตอร์จากข้อมูลปฏิสัมพันธ์ที่ผ่านมา เช่น เรียนรู้ว่าผู้ใช้ชอบสีไฟแบบใด แล้วปรับค่า RGB อัตโนมัติในการใช้งานครั้งต่อไป งานเสริมที่อยู่นอกแล็บหลักคือ Learning Preference ซึ่งต้องออกแบบกลไกการอัปเดตและบันทึกค่าที่เรียนรู้เพิ่มเติม

เวลาคาบ 180 นาที รวม emotion controller lab 50 นาที และ RGB กับ response test 30 นาที Learning preference, sound และ phone interaction เป็นงานเสริม ไม่ใช่เงื่อนไขผ่านแล็บหลัก